```java
package bankeralgorithm;

import java.util.*;

public class BankersAlgorithm {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of processes: ");
        int n = sc.nextInt();
        System.out.print("Enter number of resources: ");
        int m = sc.nextInt();

        int available[] = new int[m];
        int max[][] = new int[n][m];
        int allocation[][] = new int[n][m];
        int need[][] = new int[n][m];

        System.out.println("\nEnter Available Resources:");
        for (int i = 0; i < m; i++) {
            available[i] = sc.nextInt();
        }
        System.out.println("\nEnter Maximum Demand:");
        for (int i = 0; i < n; i++) {
            System.out.println("Process P" + i);
            for (int j = 0; j < m; j++) {
                max[i][j] = sc.nextInt();
            }
        }
        System.out.println("\nEnter Currently Allocated Resources:");
        for (int i = 0; i < n; i++) {
            System.out.println("Process P" + i);
            for (int j = 0; j < m; j++) {
                allocation[i][j] = sc.nextInt();
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                need[i][j] = max[i][j] - allocation[i][j];
            }
        }
        System.out.println("\nNeed Matrix:");
        for (int i = 0; i < n; i++) {
            System.out.print("P" + i + ": ");
```

```
    for (int j = 0; j < m; j++) {
        System.out.print(need[i][j] + " ");
    }
    System.out.println();
}
// BANKER'S ALGORITHM
int work[] = new int[m];
boolean finish[] = new boolean[n];
int safeSequence[] = new int[n];
int count = 0;
for (int i = 0; i < m; i++) {
    work[i] = available[i];
}
while (count < n) {
    boolean found = false;
    for (int i = 0; i < n; i++) {
        if (finish[i] == false) {
            boolean possible = true;
            for (int j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                    possible = false;
                    break;
                }
            }
            if (possible) {
                for (int j = 0; j < m; j++) {
                    work[j] = work[j] + allocation[i][j];
                }
                safeSequence[count] = i;
                count++;
                finish[i] = true;
                found = true;
            }
        }
    }
    if (found == false) {
        break;
    }
}
// RESULT
if (count == n) {
    System.out.println("\nSystem is in SAFE STATE.");
    System.out.print("Safe Sequence: ");
    for (int i = 0; i < n; i++) {
```

```
                System.out.print("P" + safeSequence[i]);
                if (i < n - 1) {
                    System.out.print(" -> ");
                }
            }
            System.out.println();
        } else {
            System.out.println("\nSystem is in UNSAFE STATE.");
            System.out.println("Deadlock may occur.");
        }
        sc.close();
    }
}
```